Time series data

In [2]:
```
# Step 1/5 import packages
import pandas as pd
import matplotlib.pyplot as plt
```

In [73]:
```
# Step 2/5 Reading the CSV file dataset
def Reading_time_series():
    df = pd.read_csv("Datasets/customers_data.csv")
    # showing index not set
    #print(df.index)
    df['time'] = pd.to_datetime(df['time'], errors='coerce')   # parse with offs
    df = df.set_index('time')
    #print(df.head())
    #showing index set
    #print(df.index)
    return df


df = Reading_time_series()
#df= df[df.index.year>2021]
df.info
```

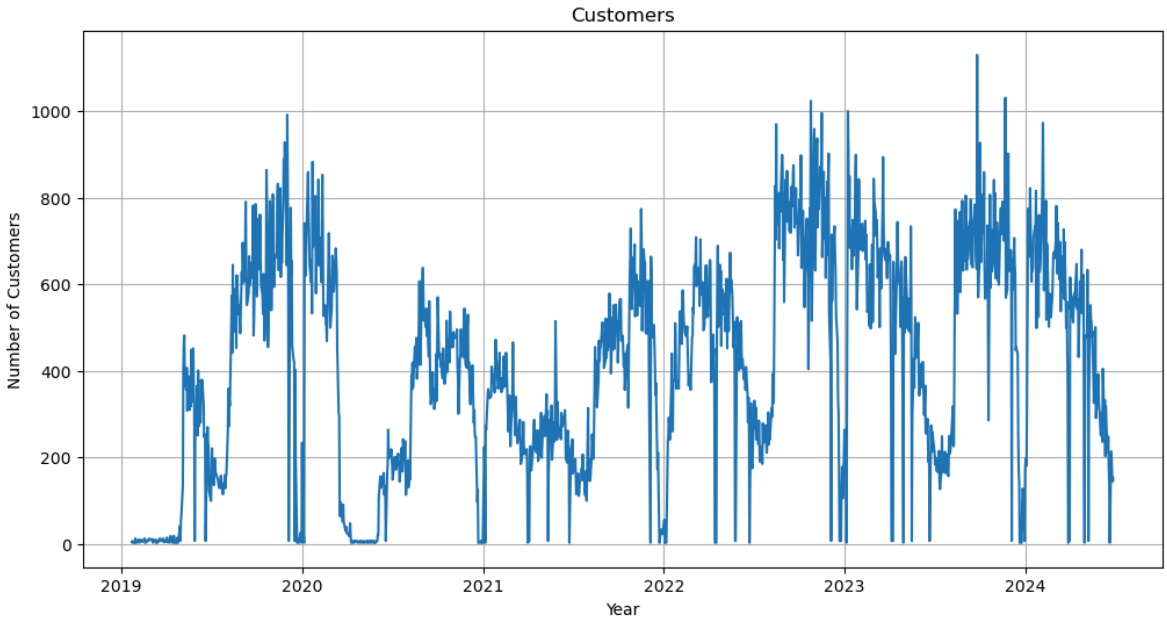
Out[73]:
```
<bound method DataFrame.info of             customers
time
2019-01-22          5
2019-01-23          7
2019-01-24          3
2019-01-27          3
2019-01-28          4
...               ...
2024-06-23        214
2024-06-24        191
2024-06-25        144
2024-06-26        155
2024-06-27        148

[1418 rows x 1 columns]>
```

In [4]:
```
# Step 3: Visualize the data
def show_data(df):
    plt.figure(figsize=(12,6))
    plt.plot(df.index, df['customers'],  linestyle='-')
    plt.title("Customer Count Over Time")
    plt.xlabel("Date")
    plt.ylabel("Number of Customers")
    plt.grid(True)
    plt.show()
#show_data(df)

plt.figure(figsize= (12,6))
plt.plot(df.index, df["customers"], linestyle ="-", marker ="")
plt.title("Customers")
plt.xlabel("Year")
plt.ylabel("Number of Customers")
```

```
plt.grid()
plt.show()
```



Temporal Features

In [15]:
```
def Make_BaseLine_features(df):
    df['year'] = df.index.year
    df['month'] = df.index.month
    df['day'] = df.index.day
    df['day_of_week'] = df.index.dayofweek
    # Add 'season' based on the quarter of the year
    #df['quarter'] = df['time'].dt.quarter
    df['quarter'] = df.index.quarter
    return df
```

Out[15]:
```
<bound method DataFrame.info of             customers  year  month  day  day_of
_week  quarter
time
2019-01-22          5  2019      1   22           1        1
2019-01-23          7  2019      1   23           2        1
2019-01-24          3  2019      1   24           3        1
2019-01-27          3  2019      1   27           6        1
2019-01-28          4  2019      1   28           0        1
...               ...   ...    ...  ...        ...      ...
2024-06-23        214  2024      6   23           6        2
2024-06-24        191  2024      6   24           0        2
2024-06-25        144  2024      6   25           1        2
2024-06-26        155  2024      6   26           2        2
2024-06-27        148  2024      6   27           3        2

[1418 rows x 6 columns]>
```

Weather data

In [12]:
```
# Function to preprocess Turku weather data
def Turku_weather_preprocess():
    # Define the file path
```

```
    turku_weather_hourly_24 = r"Datasets/Turku_Hourly_2021-2024_turkuTime_new2.c

    # Read the CSV file
    weather_df_raw = pd.read_csv(turku_weather_hourly_24, encoding='latin-1')

    # Drop unnecessary column
    weather_df_raw.drop(columns=['Observation station'], inplace=True)

    # Rename columns
    weather_df_raw = weather_df_raw.rename(columns={
        'Year': 'Year',
        'Month': 'Month',
        'Day': 'Day',
        'Time [Local time]': 'Hour',
        'Average temperature [Â°C]': 'avg_t',
        'Maximum temperature [Â°C]': 'mx_t',
        'Minimum temperature [Â°C]': 'mn_t',
        'Wind speed [m/s]': 'wnd_spd',
        'Maximum wind speed [m/s]': 'mx_wnd_spd',
        'Average wind direction [Â°]': 'av_wnd_dir',
        'Precipitation [mm]': 'precn',
        'Maximum gust speed [m/s]': 'mx_Gust_spd',
        'Average air pressure [hPa]': 'avg_air_pr'
    })

    # Convert time columns
    weather_df_raw['time'] = pd.to_datetime(weather_df_raw['Hour'], format='%H:%
    weather_df_raw['date'] = pd.to_datetime(weather_df_raw[['Year', 'Month', 'Da
    weather_df_raw['time'] = weather_df_raw['date'] + pd.to_timedelta(weather_df

    # Filter hours and drop unnecessary columns
    filtered_df = weather_df_raw[(weather_df_raw['time'].dt.hour >= 10) & (weath
    filtered_df = filtered_df.drop(columns=['Year', 'Month', 'Day', 'Hour', 'dat

    # Convert relevant columns to numeric
    numeric_columns = ['avg_t', 'mx_t', 'mn_t', 'wnd_spd', 'mx_wnd_spd', 'av_wnd
    filtered_df[numeric_columns] = filtered_df[numeric_columns].apply(pd.to_nume
    filtered_df = (filtered_df.groupby(filtered_df['time'].dt.normalize(), as_in


    return filtered_df
#w = Turku_weather_preprocess()
```

In [76]:

```
import pandas as pd
import holidays

def add_previous_day_customers(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    # yesterday's customers
    df["previous_day_customers"] = df["customers"].shift(1).fillna(0)
    return df


def add_rolling_mean_feature(df: pd.DataFrame, window: int = 5) -> pd.DataFrame:
    df = df.copy()
    # rolling mean up to yesterday (no leakage)
    df["rolling_mean"] = df["customers"].shift(1).rolling(window=window, min_per
    df["rolling_mean"] = df["rolling_mean"].fillna(df["rolling_mean"].mean())
    return df
```

```python
def add_exponential_smoothing_feature(df: pd.DataFrame, span: int = 5) -> pd.Dat
    df = df.copy()
    # exponential smoothing up to yesterday
    df["exponential_smoothing"] = df["customers"].shift(1).ewm(span=span, adjust
    df["exponential_smoothing"] = df["exponential_smoothing"].fillna(df["exponen
    return df


def add_rolling_autocorr_feature(df: pd.DataFrame, window: int = 14, lag: int =
    df = df.copy()
    # rolling autocorrelation up to yesterday
    roll_autocorr = (
        df["customers"].shift(1)  # avoid peeking at today
        .rolling(window=window, min_periods=max(lag + 1, 3))
        .apply(lambda x: x.autocorr(lag=lag), raw=False)
    )
    df["autocorrelation"] = roll_autocorr.fillna(0)  # safer default than mean
    return df

# === Holiday Feature Function ===
def add_holiday_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    if not isinstance(df.index, pd.DatetimeIndex):
        raise ValueError("DataFrame index must be a DatetimeIndex.")

    fi_holidays = holidays.Finland(years=range(2019, 2025))
    holiday_idx = pd.DatetimeIndex(pd.to_datetime(list(fi_holidays.keys())))
    d = df.index.normalize()

    df["is_holiday"] = d.isin(holiday_idx).astype(int)
    df["day_before_holiday"] = (d + pd.Timedelta(days=1)).isin(holiday_idx).asty
    df["day_after_holiday"] = (d - pd.Timedelta(days=1)).isin(holiday_idx).astyp

    return df


# === Orchestrator ===
def build_all_features(df: pd.DataFrame) -> pd.DataFrame:
    df = Make_BaseLine_features(df)
    df = add_previous_day_customers(df)
    df = add_rolling_mean_feature(df, window=5)
    df = add_exponential_smoothing_feature(df, span=5)
    df = add_rolling_autocorr_feature(df, window=14, lag=1)
    df = add_holiday_features(df)
    return df
################################################################################

# === Prepare Data ===
df = Reading_time_series()
df = build_all_features(df)   # <- your corrected pipeline
df = df.dropna()

target = "customers"
features = [col for col in df.columns if col != target]
```

Out[76]:
```
['year',
 'month',
 'day',
 'day_of_week',
 'quarter',
 'previous_day_customers',
 'rolling_mean',
 'exponential_smoothing',
 'autocorrelation',
 'is_holiday',
 'day_before_holiday',
 'day_after_holiday']
```

In [77]:
```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from xgboost import XGBRegressor
import numpy as np

# === Prepare Data ===
df = Reading_time_series()
df = build_all_features(df)   # <- your corrected pipeline
df = df.dropna()

target = "customers"
features = [col for col in df.columns if col != target]

X = df[features]
y = df[target]

# === Train/Test Split (time-aware, no shuffle) ===
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

# === Model ===
model = XGBRegressor(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

# === Train ===
model.fit(X_train, y_train)

# === Predict ===
y_pred = model.predict(X_test)

# === Evaluate ===
mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae  = mean_absolute_error(y_test, y_pred)
r2   = r2_score(y_test, y_pred)

print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
```

```
print(f"MAE:  {mae:.2f}")
print(f"R²:   {r2:.2f}")
```

```
MSE:  14640.44
RMSE: 121.00
MAE:  97.69
R²:   0.74
```

In [78]:

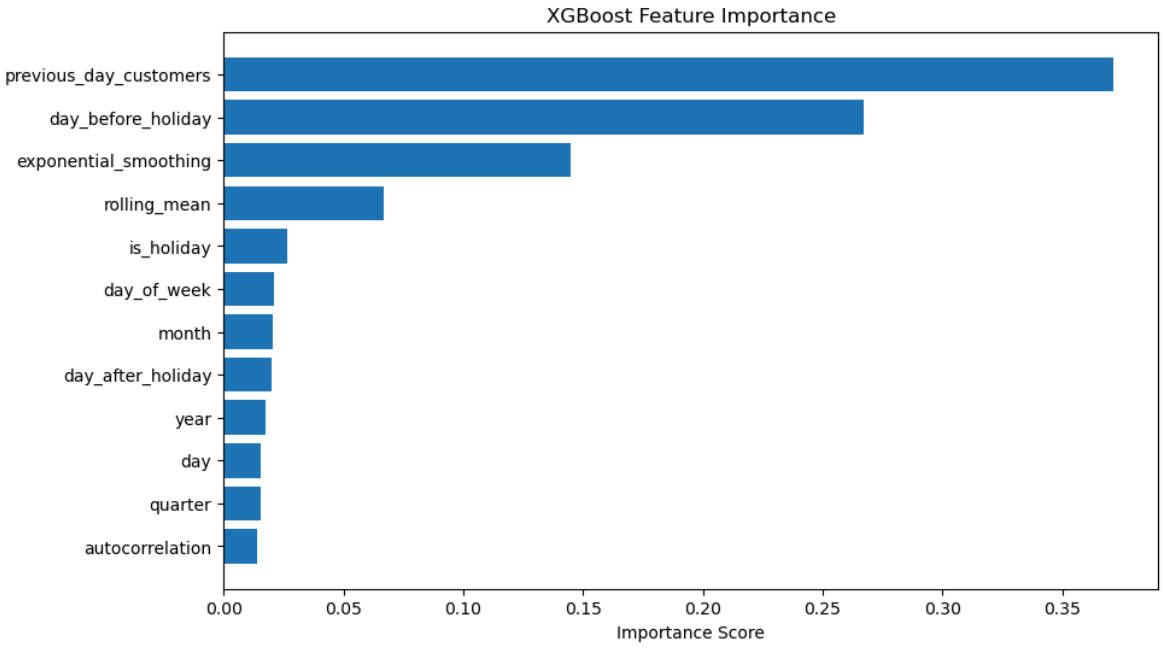
```
import pandas as pd
import matplotlib.pyplot as plt

# Get feature importance scores
importances = model.feature_importances_
feature_importance = pd.DataFrame({
    "feature": X.columns,
    "importance": importances
}).sort_values(by="importance", ascending=False)

print(feature_importance)

# Optional: plot
plt.figure(figsize=(10, 6))
plt.barh(feature_importance["feature"], feature_importance["importance"])
plt.gca().invert_yaxis()
plt.title("XGBoost Feature Importance")
plt.xlabel("Importance Score")
plt.show()
```

	feature	importance
5	previous_day_customers	0.371411
10	day_before_holiday	0.266997
7	exponential_smoothing	0.144880
6	rolling_mean	0.066692
9	is_holiday	0.026485
3	day_of_week	0.020775
1	month	0.020569
11	day_after_holiday	0.020181
0	year	0.017379
2	day	0.015588
4	quarter	0.015300
8	autocorrelation	0.013742



In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Feature importance dictionary (your values)
importances = {
    "previous_day_customers": 0.371411,
    "day_before_holiday": 0.266997,
    "exponential_smoothing": 0.144880,
    "rolling_mean": 0.066692,
    "is_holiday": 0.026485,
    "day_of_week": 0.020775,
    "month": 0.020569,
    "day_after_holiday": 0.020181,
    "year": 0.017379,
    "day": 0.015588,
    "quarter": 0.015300,
    "autocorrelation": 0.013742
}

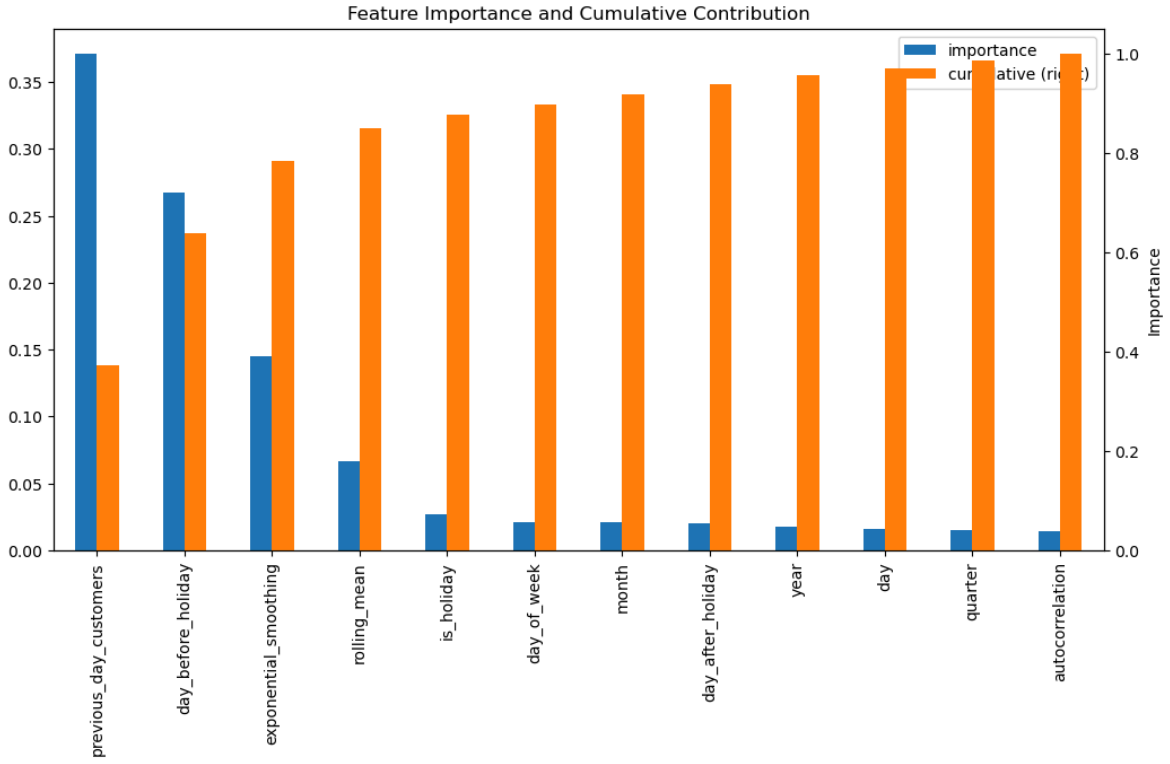
# Sort by importance
feat_df = pd.DataFrame.from_dict(importances, orient='index', columns=['importan
feat_df = feat_df.sort_values(by='importance', ascending=False)

# Cumulative importance
feat_df['cumulative'] = feat_df['importance'].cumsum()

print(feat_df)

# Plot
feat_df[['importance','cumulative']].plot(kind="bar", figsize=(12,6), secondary_
plt.title("Feature Importance and Cumulative Contribution")
plt.ylabel("Importance")
plt.show()
```

```
                      importance  cumulative
previous_day_customers    0.371411    0.371411
day_before_holiday        0.266997    0.638408
exponential_smoothing     0.144880    0.783288
rolling_mean              0.066692    0.849980
is_holiday                0.026485    0.876465
day_of_week               0.020775    0.897240
month                     0.020569    0.917809
day_after_holiday         0.020181    0.937990
year                      0.017379    0.955369
day                       0.015588    0.970957
quarter                   0.015300    0.986257
autocorrelation           0.013742    0.999999
```

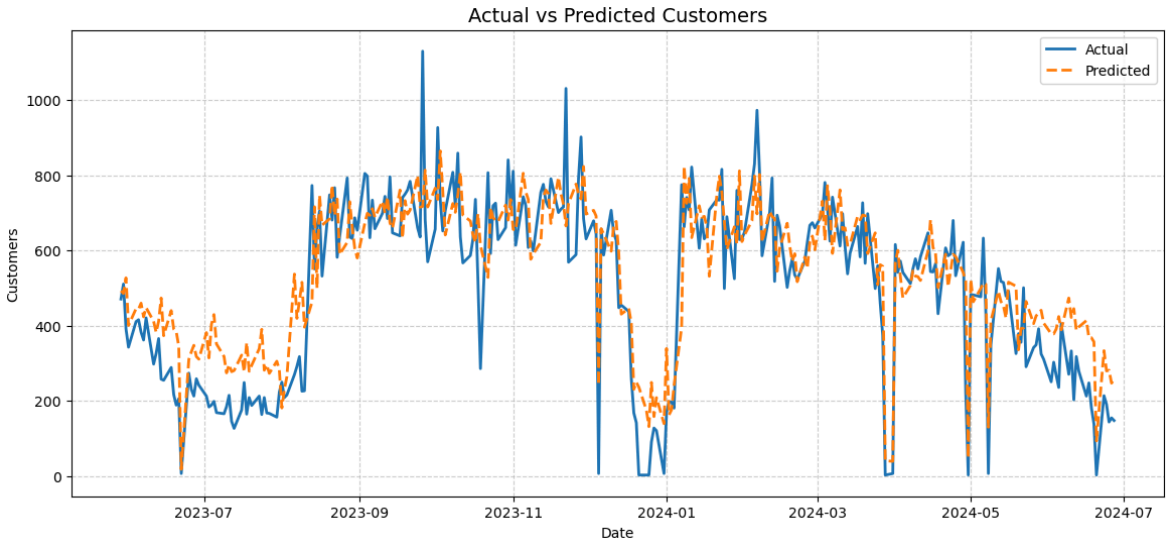


In [79]:
```
import matplotlib.pyplot as plt

# Align series with the test index
results = y_test.to_frame(name="actual").copy()
results["predicted"] = y_pred
results.index = X_test.index  # keep the datetime index aligned

# Plot
plt.figure(figsize=(14, 6))
plt.plot(results.index, results["actual"], label="Actual", linewidth=2)
plt.plot(results.index, results["predicted"], label="Predicted", linewidth=2, li

plt.title("Actual vs Predicted Customers", fontsize=14)
plt.xlabel("Date")
plt.ylabel("Customers")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```



In [32]:
```
#import sys, os
#sys.path.append(os.path.abspath("predicting_customers/lib/python3.10/site-packa

#import holidays
#print(holidays.__file__)  # sanity check


#import holidays
```

```
/scratch/project_2014453/TimeSeries_Lecture/predicting_customers/lib/python3.10/s
ite-packages/holidays/__init__.py
```